

MODULE 5

Run-Time Environments: Source Language issues, Storage organization, Storage- allocation strategies.

Intermediate Code Generation (ICG): Intermediate languages – Graphical representations, Three-Address code, Quadruples, Triples. Assignment statements, Boolean expressions.

5.1 RUN-TIME ENVIRONMENTS

A translation needs to relate the static source text of a program to the dynamic actions that must occur at runtime to implement the program. The program consists of names for procedures, identifiers etc., that require mapping with the actual memory location at runtime.

Runtime environment is a state of the target machine, which may include software libraries, environment variables, etc., to provide services to the processes running in the system.

5.1.1 SOURCE LANGUAGE ISSUES

Procedure

A *procedure* definition is a declaration that associates an identifier with a statement. The identifier is *procedure* name, and statement is the *procedure* body.

For example, the following definition of procedure named *readarray*

```
procedure readarray;  
  var i : integer;  
  
  begin  
  
    for i := 1 to 9 do read(a[i])  
  
  end;
```

When a procedure name appears with in an executable statement, the procedure is said to be *called* at that point.

Activation Tree

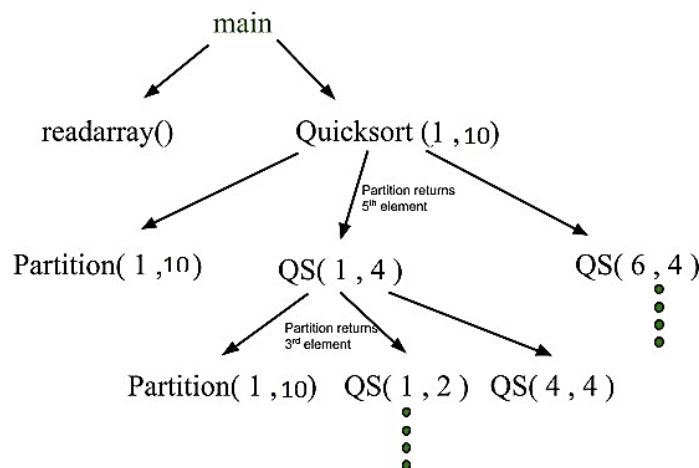
- ✚ Each execution of procedure is referred to as an activation of the procedure. Lifetime of an activation is the sequence of steps present in the execution of the procedure.
- ✚ If 'a' and 'b' be two procedures, then their activations will be non-overlapping (when one is called after other) or nested (nested procedures).
- ✚ A procedure is recursive if a new activation begins before an earlier activation of the same procedure has ended. An activation tree shows the way control enters and leaves, activations.
- ✚ Properties of activation trees are :-
 - ❖ Each node represents an activation of a procedure.
 - ❖ The root shows the activation of the main function.
 - ❖ The node for procedure 'x' is the parent of node for procedure 'y' if and only if the control flows from procedure x to procedure y.

EXAMPLE

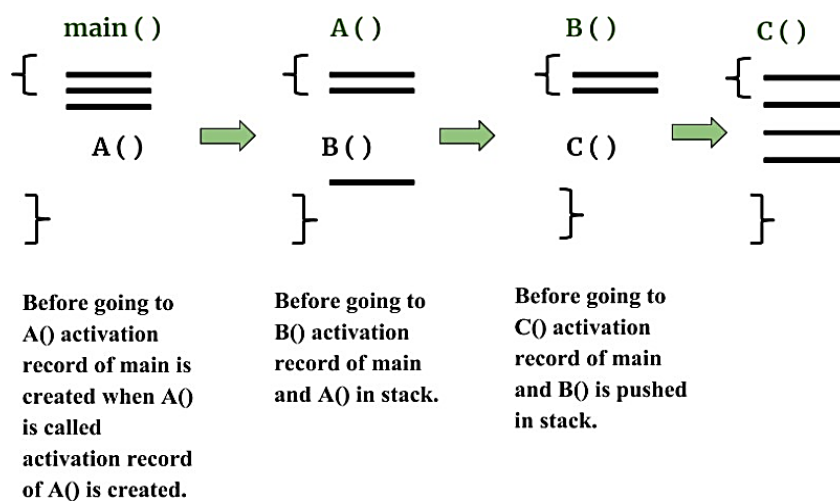
Consider the following program of quicksort

```
main()
{
    readarray();
    quicksort(1,10);
}

quicksort(int m, int n)
{
    int i= partition(m,n);
    quicksort(m,i-1);
    quicksort(i+1,n);
}
```



- First main function as root then main calls readarray and quicksort.
- Quicksort in turn calls partition and quicksort again. The flow of control in a program corresponds to the depth first traversal of activation tree which starts at the root.



Control Stack

- ✚ Control stack or runtime stack is used to keep track of the live procedure activations i.e the procedures whose execution have not been completed.
- ✚ A procedure name is pushed on to the stack when it is called (activation begins) and it is popped when it returns (activation ends).
- ✚ Information needed by a single execution of a procedure is managed using an activation record.
- ✚ When a procedure is called, an activation record is pushed into the stack and as soon as the control returns to the caller function the activation record is popped.
- ✚ Then the contents of the control stack are related to paths to the root of the activation tree. When node n is at the top of the control stack, the stack contains the nodes along the path from n to the root.
- ✚ Consider the above activation tree, when `quicksort(4,4)` gets executed, the contents of control stack were `main()` `quicksort(1,10)` `quicksort(1,4)`, `quicksort(4,4)`

<code>quicksort(4,4)</code>
<code>Quicksort(1,4)</code>
<code>Quicksort(1.10)</code>
<code>Main()</code>

The Scope Of Declaration

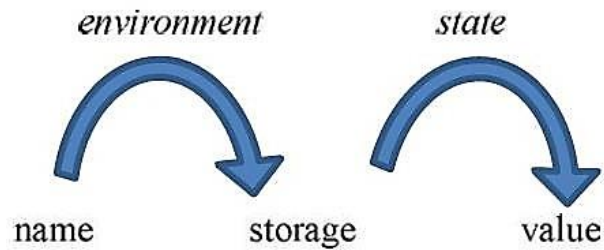
- ✚ A declaration is a syntactic construct that associates information with a name. Declaration may be explicit such as

`var i : integer;`

or may be explicit. The portion of program to which a declaration applies is called the **scope** of that declaration.

Binding Of Names

- ✚ Even if each name is declared once in a program, the same name may denote different data object at run time. "Data objects" corresponds to a storage location that hold values.
- ✚ The term *environment* refers to a function that maps a name to a storage location.
- ✚ The term *state* refers to a function that maps a storage location to the value held there.

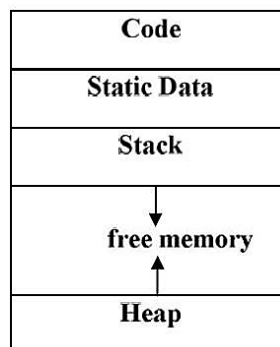


- When an environment associates storage location s with a name x , we say that x is bounds to s . This association is referred to as a binding of x .

5.1.2 STORAGE ORGANIZATION

- The executing target program runs in its own logical address space in which each program value has a location
- The management and organization of this logical address space is shared between the compiler, operating system and target machine. The operating system maps the logical address into physical addresses, which are usually spread through memory.

Typical subdivision of run time memory.



- Code area:** used to store the generated executable instructions, memory locations for the code are determined at compile time
- Static Data Area:** Is the locations of data that can be determined at compile time
- Stack Area:** Used to store the data object allocated at runtime. eg. Activation records
- Heap:** Used to store other dynamically allocated data objects at runtime (for ex: malloc)
- This runtime storage can be subdivided to hold the different components of an existing system
 - Generated executable code
 - Static data objects
 - Dynamic data objects-heap
 - Automatic data objects-stack

Activation Records

- ✚ It is LIFO structure used to hold information about each instantiation.
- ✚ Procedure calls and returns are usually managed by a run time stack called control stack.
- ✚ Each live activation has an activation record on control stack, with the root of the activation tree at the bottom, the latter activation has its record at the top of the stack
- ✚ The contents of the activation record vary with the language being implemented.
- ✚ The diagram below shows the contents of an activation record.
- ✚ The purpose of the fields of an activation record is as follows, starting from the field for temporaries.
 1. Temporary values, such as those arising in the evaluation of expressions, are stored in the field for temporaries.
 2. The field for local data holds data that is local to an execution of a procedure.
 3. The field for saved machine status holds information about the state of the machine just before the procedure is called. This information includes the values of the program counter and machine registers that have to be restored when control returns from the procedure.
 4. The optional access link is used to refer to nonlocal data held in other activation records.
 5. The optional control link points to the activation record of the caller
 6. The field for actual parameters is used by the calling procedure to supply parameters to the called procedure.
 7. The field for the returned value is used by the called procedure to return a value to the calling procedure, Again, in practice this value is often returned in a register for greater efficiency.

Returned value
Actual parameters
Optional control link
Optional access link
Saved machine status
Local data
temporaries

General Activation Record

5.1.3 STORAGE ALLOCATION STRATEGIES

- ✚ The different storage allocation strategies are:

Static allocation - lays out storage for all data objects at compile time

Stack allocation - manages the run-time storage as a stack.

Heap allocation - allocates and deallocates storage as needed at run time from a data area known as heap.

Static Allocation

- ✚ In static allocation, names bound to storage as the program is compiled, so there is no need for a run-time support package.
- ✚ Since the bindings do not change at runtime, every time a procedure activated, its run-time, names bounded to the same storage location.
- ✚ Therefore, values of local names retained across activations of a procedure. That is when control returns to a procedure the value of the local are the same as they were when control left the last time.
- ✚ From the type of a name, the compiler decides amount of storage for the name and decides where the activation records go. At compile time, we can fill in the address at which the target code can find the data it operates on.

Stack Allocation

- ✚ All compilers for languages that use procedures, functions or methods as units of user functions define actions manage at least part of their runtime memory as a stack run-time stack.
- ✚ Each time a procedure called, space for its local variables is pushed onto a stack, and when the procedure terminates, space popped off from the stack

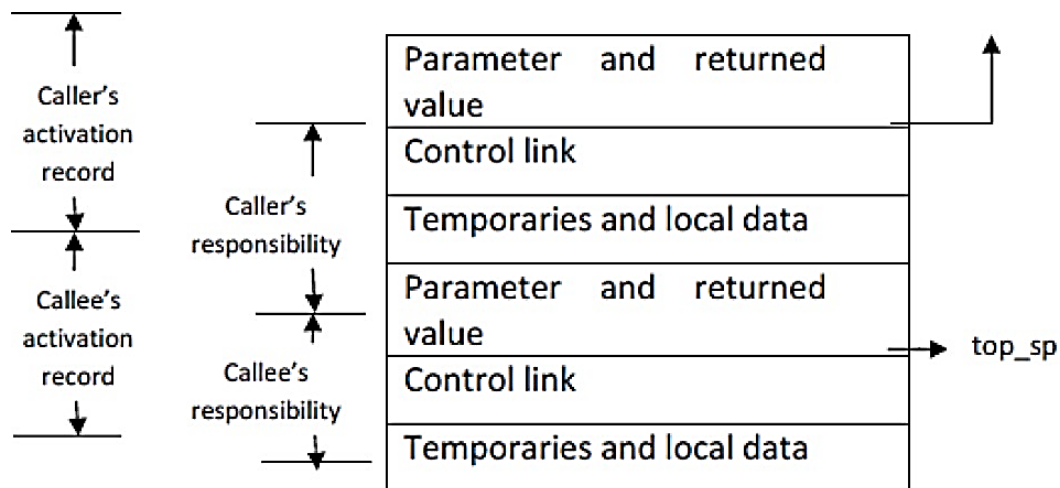
Calling Sequences

- ✚ Procedures called implemented in what is called as calling sequence, which consists of code that allocates an activation record on the stack and enters information into its fields.
- ✚ A return sequence is similar to code to restore the state of a machine so the calling procedure can continue its execution after the call.
- ✚ The code is calling sequence of often divided between the calling procedure (caller) and a procedure is calls (callee)(callee).
- ✚ When designing calling sequences and the layout of activation record, the following principles are helpful:

1. Value communicated between caller and callee generally placed at the caller beginning of the callee's activation record, so they as close as possible to the caller's activation record.
2. Fixed length items generally placed in the middle. Such items typically include the control link, the access link, and the machine status field.
3. Items whose size may not be known early enough placed at the end of the activation record.
4. We must locate the top of the stack pointer judiciously. A common approach is to have it point to the end of fixed length fields in the activation is to have it point to fix the end of fixed length fields in the activation record. Fixed length data can then be accessed by fixed offsets, known to the **intermediate code generator**, relative to the top of the stack pointer.

✚ The calling sequence and its division between caller and callee are as follows:

1. The caller evaluates the actual parameters.
2. The caller stores a return address and the old value of top_sp into the callee's activation record. The caller then increments the top_sp to the respective positions.
3. The callee-saves the register values and other status information.
4. The callee initializes its local data and begins execution.

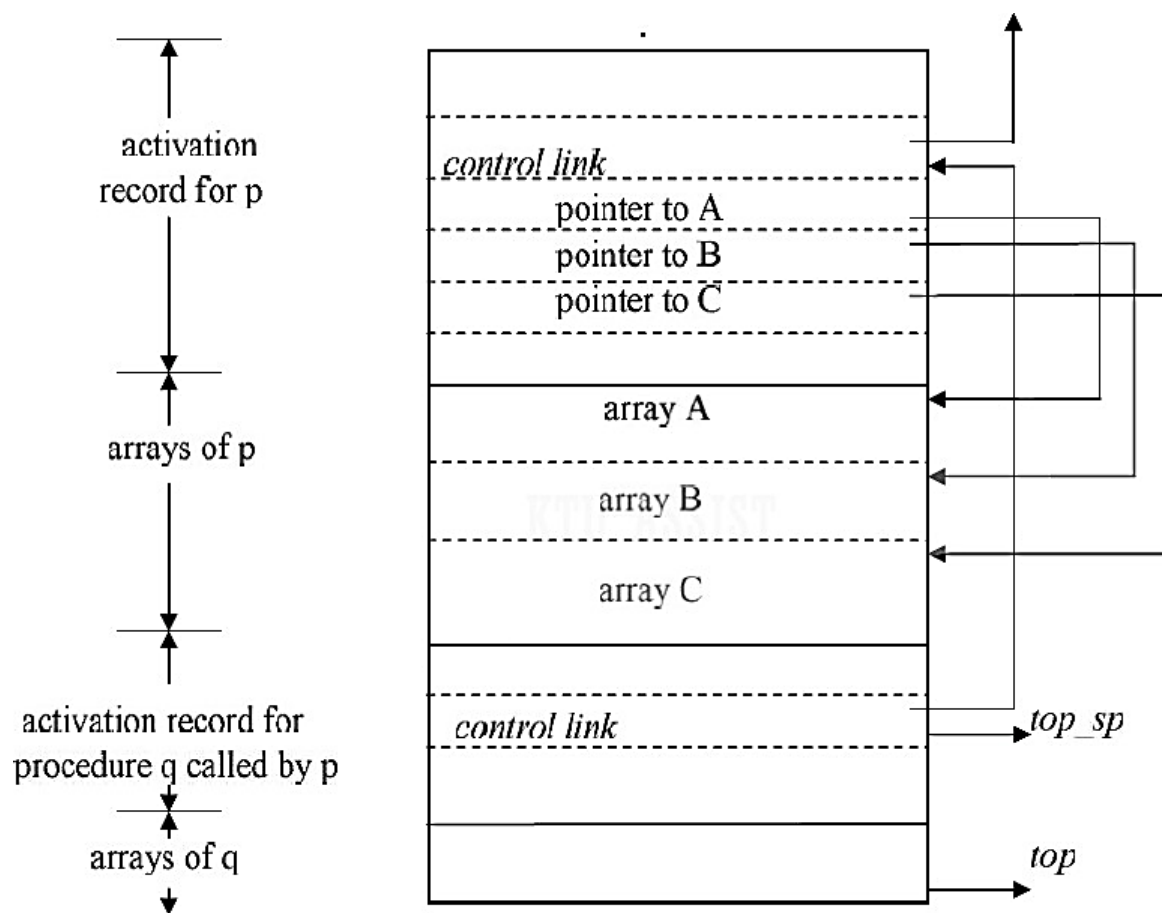


✚ A suitable, corresponding return sequence is:

1. The callee places the return value next to the parameters.
2. Using the information in the machine status field, the callee restores top_sp and other registers, and then branches to the return address that the caller placed in the status field.
3. Although top_sp has been decremented, the caller knows where the return value is, relative to the current value of top_sp; the caller, therefore, may use that value.

Variable length data on the stack

- ✚ The run-time memory-management system must deal frequently with the allocation of space for **objects the sizes of which are not known** at compile time, but which are local to a procedure and thus **may be allocated on the stack**.
- ✚ In modern languages, objects whose **size cannot be determined at compile time** are **allocated space in the heap**
- ✚ However, it is also **possible to allocate objects, arrays, or other structures of unknown size on the stack**.
- ✚ We **avoid the expense of garbage collecting** their space. Note that the stack can be used only for an object if it is local to a procedure and **becomes inaccessible when the procedure returns**.
- ✚ A common strategy for allocating variable-length arrays is shown in following figure



Access to dynamically allocated arrays

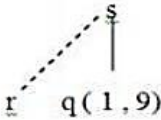
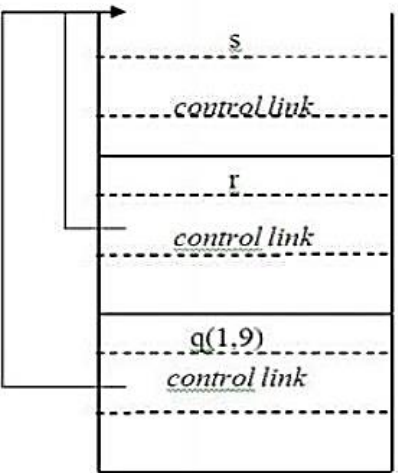
Heap Allocation

Stack allocation strategy cannot be used if either of the following is possible :

1. The values of local names must be retained when an activation ends.
2. A called activation outlives the caller.

Heap allocation parcels out pieces of contiguous storage, as needed for activation records or other objects.

Pieces may be deallocated in any order, so over the time the heap will consist of alternate areas that are free and in use.

Position in the activation tree	Activation records in the heap	Remarks
		Retained activation record for r

Records for live activations need not be adjacent in heap

The record for an activation of procedure r is retained when the activation ends.

Therefore, the record for the new activation q(1 , 9) cannot follow that for s physically.

If the retained activation record for r is deallocated, there will be free space in the heap between the activation records for s and q.

5.2 INTERMEDIATE CODE GENERATION (ICG)

In compiler, the front-end translates a source program into an intermediate representation from which the back end generates target code.

Need For ICG

1. If a compiler translates the source language to its target machine language without generating IC, then for each new machine, a full native compiler is required.
2. IC eliminates the need of a new full compiler for every machine by keeping the analysis portion for all the compilers.
3. Synthesis part of back end depends on the target machine.

2 important things:

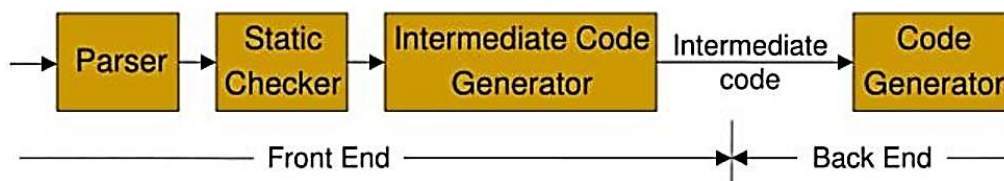
- IC Generation process should not be very complex
- It shouldn't be difficult to produce the target program from the intermediate code.



A source program can be translated directly into the target language, but some benefits of using intermediate form are:

- Retargeting is facilitated: a compiler for a different machine can be created by attaching a Back-end (which generate Target Code) for the new machine to an existing Front-end (which generate Intermediate Code).
- A machine Independent Code-Optimizer can be applied to the Intermediate Representation.

Logical Structure of a Compiler Front End



5.2.1 INTERMEDIATE LANGUAGES

The most commonly used intermediate representations were:-

- Syntax Tree
- DAG (Direct Acyclic Graph)
- Postfix Notation
- 3 Address Code

5.2.1.1 GRAPHICAL REPRESENTATION

Includes both

- Syntax Tree
- DAG (Direct Acyclic Graph)

Syntax Tree Or Abstract Syntax Tree (AST)

- 🚦 Graphical Intermediate Representation
- 🚦 Syntax Tree depicts the hierarchical structure of a source program.
- 🚦 Syntax tree (AST) is a condensed form of parse tree useful for representing language constructs.

EXAMPLE

Parse tree and syntax tree for $3 * 5 + 4$ as follows.

Grammar

$E \rightarrow E + T$

$E \rightarrow E - T$

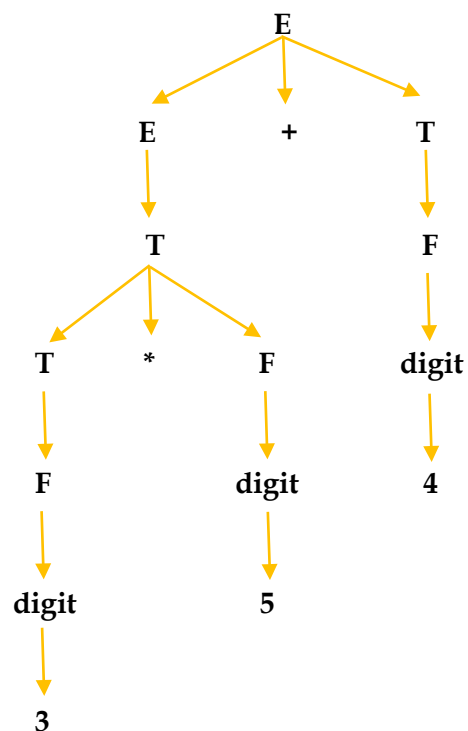
$E \rightarrow T$

$T \rightarrow T * F$

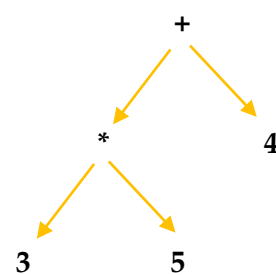
$T \rightarrow F$

$F \rightarrow \text{digit}$

Parse Tree



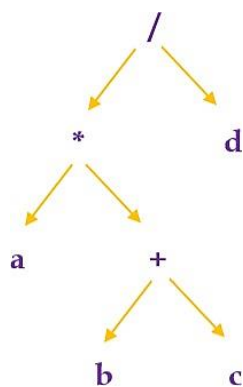
Syntax Tree



Parse Tree VS Syntax Tree

<u>Parse Tree</u>	<u>Syntax Tree</u>
A parse tree is a graphical representation of a replacement process in a derivation	A syntax tree (AST) is a condensed form of parse tree
Each interior node represents a grammar rule	Each interior node represents an operator
Each leaf node represents a terminal	Each leaf node represents an operand
Parse tree represent every detail from the real syntax	Syntax tree does not represent every detail from the real syntax Eg : No parenthesis

Syntax tree for $a * (b + c) / d$



Constructing Syntax Tree For Expression

- ✚ Each node in a syntax tree can be implemented in arecord with several fields.
- ✚ In the node of an operator, one field contains operator and remaining field contains pointer to the nodes for the operands.
- ✚ When used for translation, the nodes in a syntax tree may contain addition of fields to hold the values of attributes attached to the node.
- ✚ Following functions are used to create syntax tree
 1. **mknode(op,left,right)**: creates an operator node with label op and two fields containing pointers to left and right.
 2. **mkleaf(id,entry)**: creates an identifier node with label id and a field containing entry, a pointer to the symbol table entry for identifier
 3. **mkleaf(num,val)**: creates a number node with label num and a field containing val, the value of the number.
- ✚ Such functions return a pointer to a newly created node.

EXAMPLE

a - 4 + c

The tree is constructed
bottom up

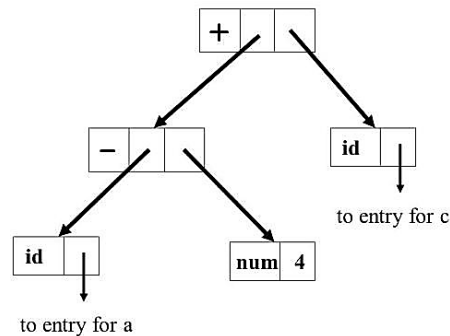
$P_1 = \text{mkleaf}(\text{id}, \text{entry } a)$

$P_2 = \text{mkleaf}(\text{num}, 4)$

$P_3 = \text{mknode}(-, P_1, P_2)$

$P_4 = \text{mkleaf}(\text{id}, \text{entry } c)$

$P_5 = \text{mknode}(+, P_3, P_4)$



Syntax Tree

Syntax directed definition

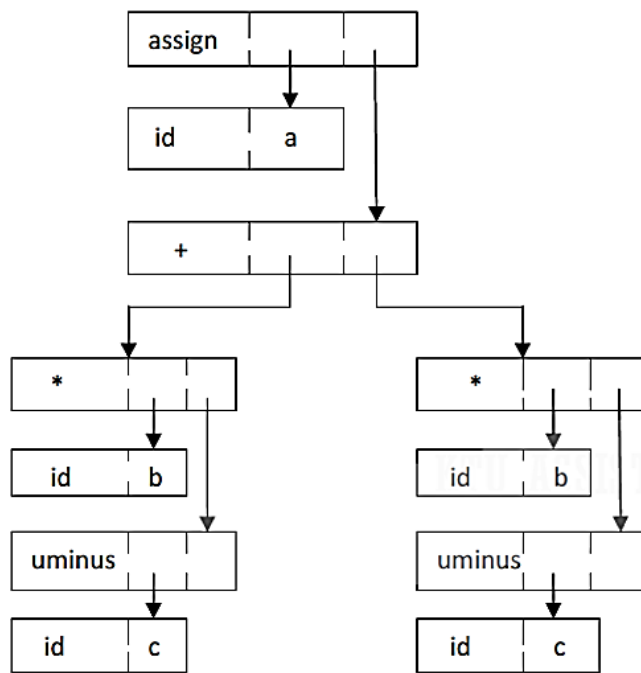
- Syntax trees for assignment statements are produced by the syntax-directed definition.
- Non terminal S generates an assignment statement.
- The two binary operators + and * are examples of the full operator set in a typical language. Operator associates and precedences are the usual ones, even though they have not been put into the grammar. This definition constructs the tree from the input $a:=b* -c + b* -c$

PRODUCTION	SEMANTIC RULE
$S \rightarrow \text{id} := E$	$S.\text{nptr} := \text{mknode}(\text{'assign'}, \text{mkleaf}(\text{id}, \text{id.place}), E.\text{nptr})$
$E \rightarrow E_1 + E_2$	$E.\text{nptr} := \text{mknode}(\text{'+'}, E_1.\text{nptr}, E_2.\text{nptr})$
$E \rightarrow E_1 * E_2$	$E.\text{nptr} := \text{mknode}(\text{'*'}, E_1.\text{nptr}, E_2.\text{nptr})$
$E \rightarrow - E_1$	$E.\text{nptr} := \text{mknode}(\text{'uminus'}, E_1.\text{nptr})$
$E \rightarrow (E_1)$	$E.\text{nptr} := E_1.\text{nptr}$
$E \rightarrow \text{id}$	$E.\text{nptr} := \text{mkleaf}(\text{id}, \text{id.place})$

Syntax-directed definition to produce syntax trees for assignment statements

- The token id has an attribute *place* that points to the symbol-table entry for the identifier.
- A symbol-table entry can be found from an attribute **id.name**, representing the lexeme associated with that occurrence of id.

- ✚ If the lexical analyser holds all lexemes in a single array of characters, then attribute name might be the index of the first character of the lexeme.
- ✚ Two representations of the syntax tree are as follows.



(a)

0	id	b	
1	id	c	
2	uminus	1	
3	*	0	2
4	id	b	
5	id	c	
6	uminus	5	
7	*	4	6
8	+	3	7
9	id	a	
10	assign	9	8

(b)

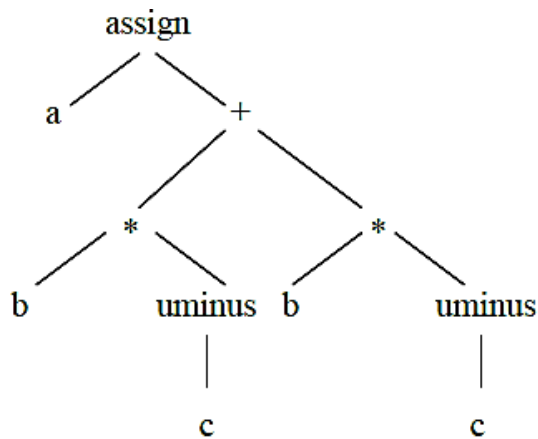
- ✚ In (a), each node is represented as a record with a field for its operator and additional fields for pointers to its children.
- ✚ In Fig (b), nodes are allocated from an array of records and the index or position of the node serves as the pointer to the node.
- ✚ All the nodes in the syntax tree can be visited by following pointers, starting from the root at position 10.

Direct Acyclic Graph (DAG)

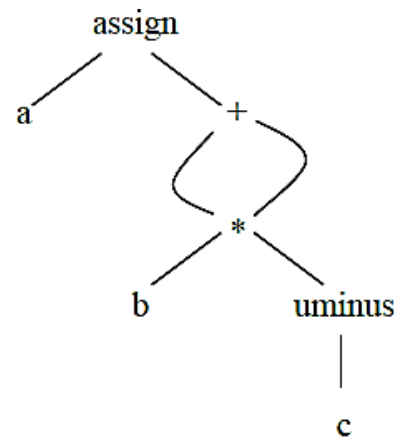
- ✚ Graphical Intermediate Representation
- ✚ Dag also gives the hierarchical structure of source program but in a more compact way because common sub expressions are identified.

EXAMPLE

$a = b * -c + b * -c$



(a) Syntax tree



(b) Dag

Postfix Notation

- Linearized representation of syntax tree
- In postfix notation, each operator appears immediately after its last operand.
- Operators can be evaluated in the order in which they appear in the string

EXAMPLE

Source String : $a := b * -c + b * -c$

Postfix String: $a \ b \ c \ \text{uminus} \ * \ b \ c \ \text{uminus} \ * \ + \ \text{assign}$

Postfix Rules

- If E is a variable or constant, then the postfix notation for E is E itself.
 - If E is an expression of the form $E_1 \text{ op } E_2$ then postfix notation for E is $E_1' \ E_2' \ \text{op}$, here E_1' and E_2' are the postfix notations for E_1 and E_2 , respectively
 - If E is an expression of the form (E) , then the postfix notation for E is the same as the postfix notation for E.
 - For unary operation $-E$ the postfix is $E-$
- Ex: postfix notation for $9 - (5 + 2)$ is $9 \ 5 \ 2 \ + \ -$
 - Postfix notation of an infix expression can be obtained using stack

5.2.1.2 THREE-ADDRESS CODE

- ✚ In Three address statement, at most 3 addresses are used to represent any statement.
- ✚ The reason for the term “three address code” is that each statement contains 3 addresses at most. Two for the operands and one for the result.

General Form Of 3 Address Code

$$a = b \text{ op } c$$

where,

a, b, c are the operands that can be names, constants or compiler generated temporaries.

op represents operator, such as fixed or floating point arithmetic operator or a logical operator on Boolean valued data. Thus a source language expression like $x + y * z$ might be translated into a sequence

$$t_1 := y * z$$

$$t_2 := x + t_1 \quad \text{where, } t_1 \text{ and } t_2 \text{ are compiler generated temporary names.}$$

Advantages Of Three Address Code

- ❖ The unraveling of complicated arithmetic expressions and of nested flow-of-control statements makes three-address code desirable for target code generation and optimization.
- ❖ The use of names for the intermediate values computed by a program allows three-address code to be easily rearranged - unlike postfix notation.

Three-address code is a linearized representation of a syntax tree or a DAG in which explicit names correspond to the interior nodes of the graph.

Three Address Code corresponding to the syntax tree and DAG given above (page no:)

$$t_1 := -c$$

$$t_2 := b * t_1$$

$$t_3 := -c$$

$$t_4 := b * t_3$$

$$t_5 := t_2 + t_4$$

$$a := t_5$$

(a) Code for the syntax tree

$$t_1 := -c$$

$$t_2 := b * t_1$$

$$t_3 := t_2 + t_2$$

$$a := t_3$$

(b) Code for the dag

Types of Three-Address Statements

1. Assignment statements

$x := y \text{ op } z$, where op is a binary arithmetic or logical operation.

2. Assignment instructions

$x := \text{op } y$, where op is a unary operation. Essential unary operations include unary minus, logical negation, shift operators, and conversion operators that for example, convert a fixed-point number to a floating-point number.

3. Copy statements

$x := y$ where the value of y is assigned to x.

4. Unconditional jump

goto L The three-address statement with label L is the next to be executed

5. Conditional jump

if $x \text{ relop } y$ goto L This instruction applies a relational operator ($<$, $=$, $>$, etc.) to x and y , and executes the statement with label **L** next if x stands in relation **relop** to y . If not, the three-address statement following **if $x \text{ relop } y$ goto L** is executed next, as in the usual sequence.

6. Procedural call and return

param x and **call p, n** for procedure calls and **return y** , where y representing a returned value is optional. Their typical use is as the sequence of three-address statements

```
param  $x_1$ 
param  $x_2$ 
... ..
param  $x_n$ 
call  $p, n$ 
```

generated as part of the call procedure **$p(x_1, x_2, \dots, x_n)$** . The integer **n** indicating the number of actual-parameters in "**call p, n** " is not redundant because calls can be nested.

7. Indexed Assignments

Indexed assignments of the form **$x = y[i]$ or $x[i] = y$**

8. Address and pointer assignments

Address and pointer operator of the form **$x := \&y$, $x := *y$ and $*x := y$**

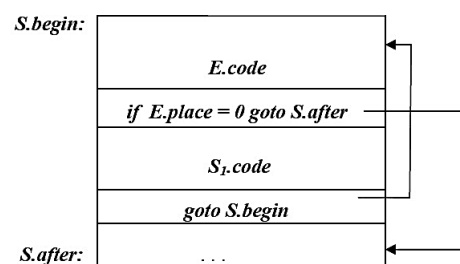
Syntax-Directed Translation Into Three-Address Code

- When three-address code is generated, temporary names are made up for the interior nodes of a syntax tree. for example $id := E$ consists of code to evaluate E into some temporary t , followed by the assignment $id.place := t$.
- Given input $a := b * -c + b + -c$, it produces the three address code in given above (page no:) The synthesized attribute **S.code** represents the three address code for the assignment S . The nonterminal E has two attributes:
 - $E.place$ the name that will hold the value of E , and
 - $E.code$. the sequence of three-address statements evaluating E .

Syntax-directed definition to produce three-address code for assignments.

PRODUCTION	SEMANTIC RULES
$S \rightarrow id := E$	$S.code := E.code \parallel gen(id.place := E.place)$
$E \rightarrow E_1 + E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel gen(E.place := E_1.place '+' E_2.place)$
$E \rightarrow E_1 * E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel gen(E.place := E_1.place '*' E_2.place)$
$E \rightarrow - E_1$	$E.place := newtemp;$ $E.code := E_1.code \parallel gen(E.place := 'uminus' E_1.place)$
$E \rightarrow (E_1)$	$E.place := E_1.place;$ $E.code := E_1.code$
$E \rightarrow id$	$E.place := id.place;$ $E.code := ''$

Semantic rule generating code for a while statement



PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{while } E \text{ do } S_1$	$S.begin := newlabel;$ $S.after := newlabel;$ $S.code := gen(S.begin ':') \parallel$ $E.code \parallel$ $gen('if' E.place '=' '0' 'goto' S.after) \parallel$ $S_1.code \parallel$ $gen('goto' S.begin) \parallel$ $gen(S.after ':')$

- ✚ The function *newtemp* returns a sequence of distinct names $t_1, t_2, \dots$ in response of successive calls. Notation $gen(x := 'y' + 'z')$ is used to represent the three address statement $x := y + z$.
- ✚ Expressions appearing instead of variables like x, y and z are evaluated when passed to *gen*, and quoted operators or operand, like '+' are taken literally.
- ✚ Flow of control statements can be added to the language of assignments. The code for $S \rightarrow \text{while } E \text{ do } S_1$ is generated using new attributes *S.begin* and *S.after* to mark the first statement in the code for E and the statement following the code for S , respectively.
- ✚ The function *newlabel* returns a new label every time is called. We assume that a nonzero expression represents true; that is when the value of E becomes zero, control leaves the while statement

Implementation Of Three-Address Statements

A three address statement is an abstract form of intermediate code. In a compiler, these statements can be implemented as records with fields for the operator and the operands. Three such, representations are

- Quadruples
- Triples
- Indirect triples

5.2.1.3 QUADRUPLES

- ✚ A quadruple is a record structure with four fields, which are *op*, *arg1*, *arg2* and *result*
- ✚ The *op* field contains an internal code for the operator. The three address statement $x := y \text{ op } z$ is represented by placing y in *arg1*, z in *arg2* and x in *result*.
- ✚ The contents of *arg1*, *arg2*, and *result* are normally pointers to the symbol table entries for the names represented by these fields. If so temporary names must be entered into the symbol table as they are created.

EXAMPLE 1

Translate the following expression to quadruple triple and indirect triple

$$a + b * c \mid e \wedge f + b * a$$

For the first construct the three address code for the expression

```

t1 = e ^ f
t2 = b * c
t3 = t2 / t1
t4 = b * a
t5 = a + t3
t6 = t5 + t4

```

Location	OP	arg1	arg2	Result
(0)	$\wedge$	e	f	t1
(1)	*	b	c	t2
(2)	/	t2	t1	t3
(3)	*	b	a	t4
(4)	+	a	t3	t5
(5)	+	t3	t4	t6

Exceptions

- ⇒ The statement **x := op y**, where op is a unary operator is represented by placing **op** in the operator field, **y** in the argument field & n in the result field. The **arg2** is not used
- ⇒ A statement like **param t1** is represented by placing **param** in the operator field and t1 in the arg1 field. Neither **arg2** not result field is used
- ⇒ Unconditional & Conditional jump statements are represented by placing the target in the result field.

5.2.1.4 TRIPLES

- ✚ In triples representation, the use of temporary variables is avoided & instead reference to instructions are made
- ✚ So three address statements can be represented by records with only there fields OP, arg1 & arg2.
- ✚ Since, there fields are used this intermediated code formal is known as triples

Advantages

- ❖ No need to use temporary variable which saves memory as well as time

Disadvantages

- ❖ Triple representation is difficult to use for optimizing compilers
- ❖ Because for optimization statements need to be suffled.
- ❖ for e.g. statement 1 can be come down or statement 2 can go up ect.
- ❖ So the reference we used in their representation will change.

EXAMPLE 1

a + b * c | e ^ f + b * a

t1 = e ^ f

t2 = b * c

t3 = t2 / t1

t4 = b * a

t5 = a + t3

t6 = t5 + t4

Location	OP	arg1	arg2
(0)	^	e	f
(1)	*	b	c
(2)	/	(1)	(0)
(3)	*	b	a
(4)	+	a	(2)
(5)	+	(4)	(3)

EXAMPLE 2

A ternary operation like $x[i] := y$ requires two entries in the triple structure while $x := y[i]$ is naturally represented as two operations.

	<i>op</i>	<i>arg1</i>	<i>arg2</i>
(0)	[] =	x	<u>i</u>
(1)	<u>assign</u>	(0)	y

$x[i] := y$

	<i>op</i>	<i>arg1</i>	<i>arg2</i>
(0)	= []	y	<u>i</u>
(1)	assign	x	(0)

$x := y[i]$

INDIRECT TRIPLES

- ✚ This representation is an enhancement over triple representation.
- ✚ It uses an additional instruction array to lead the pointer to the triples in the desired order.
- ✚ Since, it uses pointers instead of position to stage reposition the expression to produce an optimized code.

EXAMPLE 1

	Statement
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)

Location	op	arg1	arg2
(0)	^	E	f
(1)	*	B	c
(2)	/	(1)	(0)
(3)	*	B	a
(4)	+	A	(2)
(5)	+	(4)	(3)

Comparison

- ✚ When we ultimately produce the target code each temporary and programmer defined name will assign runtime memory location
- ✚ This location will be entered into symbol table entry of that data.
- ✚ Using the quadruple notation, a three address statement containing a temporary can immediately access the location for that temporary via symbol table.
- ✚ But this is not possible with triples notation.
- ✚ With quadruple notation, statements can often move around which makes optimization easier.
- ✚ This is achieved because using quadruple notation the symbol table interposes high degree of indirection between computation of a value and its use.
- ✚ With quadruple notation, if we move a statement computing x , the statement using x requires no change.
- ✚ But with triples, moving a statement that defines a temporary value requires us to change all references to that statement in $arg1$ and $arg2$ arrays. This makes triples difficult to use in optimizing compiler
- ✚ With indirect triples also, there is no such problem.
- ✚ A statement can be moved by reordering the statement list.

Space Utilization

- ✚ Quadruples and indirect triples requires same amount of space for storage (normal case).
- ✚ But if same temporary value is used more than once indirect triples can save some space. This is bcz, 2 or more entries in statement array can point to the same line of $op-arg1-arg2$ structure.
- ✚ Triples requires less space for storage compared to above 2.

✚ Quadruples

- direct access of the location for temporaries
- easier for optimization

Triples

- space efficiency

Indirect Triples

- easier for optimization
- space efficiency

PROBLEM 1

Translate the following expression to quadruple tuples & indirect tuples

$$a = b * -c + b * -c$$

Sol : - Three address code for given expression is

TAC

t1 = uniminus c

t2 = b* t1

t3 = uniminus c

t4 = b* t3

t5 = t2 + t4

Q = t5

QUADRUPLES

Location	OP	arg1	arg2	result
(0)	uniminus	c		t1
(1)	*	b	t1	t2
(3)	uniminus	c		t3
(4)	*	b	t3	t4
(5)	+	t2	t4	t5
(6)	=	t5		a

TRIPLES

Location	OP	arg1	arg2
(1)	uniminus	c	
(2)	*	b	(1)
(3)	uniminus	c	
(4)	*	b	(3)
(5)	+	(2)	(4)
(6)	=	a	(5)

INDIRECT TRIPLES

	Statements
35	(1)
36	(2)
37	(3)
38	(4)
39	(5)
40	(6)

Location	OP	arg1	arg2
(1)	uniminus	C	
(2)	*	B	(1)
(3)	uniminus	C	
(4)	*	B	(3)
(5)	+	(2)	(4)
(6)	=	A	(5)

5.2.1.5 ASSIGNMENT STATEMENTS

Translation Scheme (SDT) To Produce Three-Address Code For Assignments

Production	Semantic action
$S \rightarrow id : = E$	<pre>{ p := lookup (id.name); if p ≠ nil then emit(p ' := ' E.place) else error }</pre>
$E \rightarrow E1 + E2$	<pre>{ E.place := newtemp; emit(E.place ' := ' E1.place ' + ' E2.place) }</pre>
$E \rightarrow E1 * E2$	<pre>{ E.place := newtemp; emit(E.place ' := ' E1.place ' * ' E2.place) }</pre>
$E \rightarrow -E1$	<pre>{ E.place := newtemp; emit (E.place ' := ' 'uminus' E1.place) }</pre>
$E \rightarrow (E1)$	<pre>{ E.place := E1.place }</pre>
$E \rightarrow id$	<pre>{ p := lookup (id.name); if p ≠ nil then E.place := p else error }</pre>

emite → generate the three address code to the output file.

newtemp → return a new temporary variable.

lookup identifier → check if the id is in symbol table

EXAMPLE : Annotated Parse Tree For Generation Of TAC For Assignment Statements

$x = a * b + c + d$

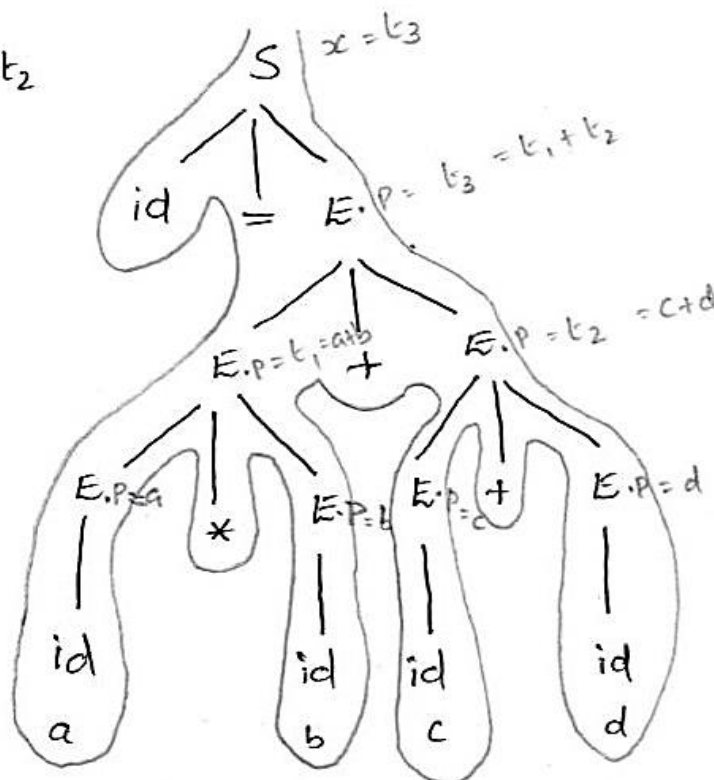
TAC

$t_1 = a * b$

$t_2 = c + d$

$t_3 = t_1 + t_2$

$x = t_3$



Syntax-directed definition to produce three-address code for assignments.

PRODUCTION	SEMANTIC RULES
$S \rightarrow id := E$	$S.code := E.code \parallel gen(id.place := E.place)$
$E \rightarrow E_1 + E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel gen(E.place := E_1.place '+' E_2.place)$
$E \rightarrow E_1 * E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel gen(E.place := E_1.place '*' E_2.place)$
$E \rightarrow - E_1$	$E.place := newtemp;$ $E.code := E_1.code \parallel gen(E.place := 'uminus' E_1.place)$
$E \rightarrow (E_1)$	$E.place := E_1.place;$ $E.code := E_1.code$
$E \rightarrow id$	$E.place := id.place;$ $E.code := ''$

5.2.1.6 BOOLEAN EXPRESSIONS

- ✚ Boolean expressions have two primary purposes.
 - They are used to compute logical values.
 - But more often they are used as conditional expressions in statements that alter the flow of control, such as if-then-else, or while-do statements.
- ✚ Boolean expressions are composed of the Boolean operators (and, or, and not) applied to elements that are Boolean variables or relational expressions.
- ✚ Relational expressions are of the form $E1 \text{ relop } E2$, where $E1$ and $E2$ are arithmetic expressions and relop can be $<$, $<=$, $=!$, $=$, $>$ or $>=$
- ✚ Here we consider Boolean expressions generated by the following grammar :

$E \rightarrow E \text{ or } E \mid E \text{ and } E \mid \text{note} \mid (E) \mid \text{id relop id} \mid \text{true} \mid \text{false}$

Methods Of Translating Boolean Expressions

- ✚ There are two principal methods of representing the value of a boolean expression. They are :
- ✚ Numerical Representation - To encode true and false numerically and to evaluate a Boolean expression analogously to an arithmetic expression. Often, 1 is used to denote true and 0 to denote false.
- ✚ Jumping Method (Short-circuit Method) - To implement Boolean expressions by flow of control, that is, representing the value of a Boolean expression by a position reached in a program. This method is particularly convenient in implementing the Boolean expressions in flow-of-control statements, such as the if-then and while-do statements.

Method 1: Numerical Representation

- ✚ Here, 1 denotes true and 0 denotes false. Expressions will be evaluated completely from left to right, in a manner similar to arithmetic expressions.

EXAMPLE

The translation for $a \text{ or } b \text{ and not } c$ will result following three-address sequence

$t1 := \text{not } c$

$t2 := b \text{ and } t1$

$t3 := a \text{ or } t2$

Translation Scheme Using A Numerical Representation For Boolean Expression

$E \rightarrow E_1 \text{ or } E_2$	$\{ E.place := newtemp;$ $emit(E.place ':=' E_1.place \text{ 'or' } E_2.place) \}$
$E \rightarrow E_1 \text{ and } E_2$	$\{ E.place := newtemp;$ $emit(E.place ':=' E_1.place \text{ 'and' } E_2.place) \}$
$E \rightarrow \text{not } E_1$	$\{ E.place := newtemp;$ $emit(E.place ':=' \text{ 'not' } E_1.place) \}$
$E \rightarrow (E_1)$	$\{ E.place := E_1.place \}$
$E \rightarrow id_1 \text{ relop } id_2$	$\{ E.place := newtemp;$ $emit(\text{ 'if' } id_1.place \text{ relop.op } id_2.place \text{ 'goto' } nextstat + 3);$ $emit(E.place ':=' '0');$ $emit(\text{ 'goto' } nextstat + 2);$ $emit(E.place ':=' '1') \}$
$E \rightarrow \text{true}$	$\{ E.place := newtemp;$ $emit(E.place ':=' '1') \}$
$E \rightarrow \text{false}$	$\{ E.place := newtemp;$ $emit(E.place ':=' '0') \}$

where the function emit() output the three address statement into the output file and nextstat() gives the index of the next three address statement in the output sequence and emit increments nextstat after producing each three address statement.

A relational expression such as $a < b$ is equivalent to the conditional statement if $a < b$ then 1 else 0 which can be translated into the three-address code sequence (let statement numbers start at 100)

```

100 if a < b goto 103
101 t := 0
102 goto 104
103 t := 1
104

```

Method 2: Jumping or Short-Circuit Code

We can also translate a boolean expression into three-address code without generating code for any of the boolean operators and without having the code necessarily evaluate the entire expression. This style of evaluation is sometimes called "short-circuit" or "jumping" code.

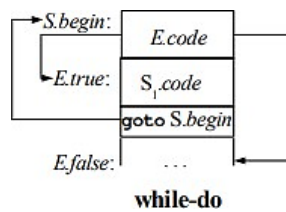
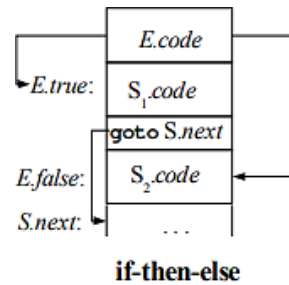
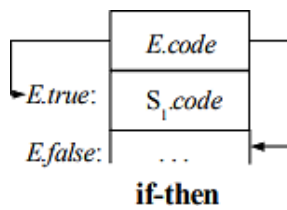
This is normally used for flow-of-control statements, such as the if-then, if-then-else and while-do statements those generated by the following grammar:

$S \rightarrow \text{if } E \text{ then } S_1$

| $\text{if } E \text{ then } S_1 \text{ else } S_2$

| $\text{while } E \text{ do } S_1$

✚ Code for if-then, if-then-else and while-do is given below:



✚ Consider the grammar

$S \rightarrow \text{if } E \text{ then } S_1$

| $\text{if } E \text{ then } S_1 \text{ else } S_2$

| $\text{while } E \text{ do } S_1$

✚ In each of these productions, E is the Boolean expression to be translated. In the translation, we assume that a three-address statement can be symbolically labeled, and that the function **newlabel** returns a new symbolic label each time it is called.

✚ With each E we associate two labels **E.true** and **E.false**. **E.true** is the label to which control flows if E is true, and **E.false** is the label to which control flows if E is false.

✚ The inherited attribute **S.next** is a label that is attached to the first three-address instruction to be executed after the code for S and another inherited attribute **S.begin** is the first instruction of S

Syntax Directed Definition for flow -of -control statements

$S \rightarrow \text{if } E \text{ then } S1$	$\{ E.\text{true} := \text{newlabel};$ $E.\text{false} := S.\text{next};$ $S1.\text{next} := S.\text{next};$ $S.\text{code} := E.\text{code} \mid \mid \text{gen}(E.\text{true} ':') \mid \mid S1.\text{code} \}$
$S \rightarrow \text{if } E \text{ then } S1 \text{ else } S2$	$\{ E.\text{true} := \text{newlabel};$ $E.\text{false} := \text{newlabel};$ $S1.\text{next} := S.\text{next};$ $S2.\text{next} := S.\text{next};$ $S.\text{code} := E.\text{code} \mid \mid \text{gen}(E.\text{true} ':') \mid \mid S1.\text{code}$ $\mid \mid \text{gen}(\text{'goto' } S.\text{next}) \mid \mid \text{gen}(E.\text{false} ':') \mid \mid S2.\text{code} \}$
$S \rightarrow \text{while } E \text{ do } S1$	$\{ S.\text{begin} := \text{newlabel};$ $E.\text{true} := \text{newlabel};$ $E.\text{false} := S.\text{next};$ $S1.\text{next} := S.\text{begin};$ $S.\text{code} := \text{gen}(S.\text{begin} ':') \mid \mid E.\text{code} \mid \mid \text{gen}(E.\text{true} ':')$ $\mid \mid S1.\text{code} \mid \mid \text{gen}(\text{'goto' } S.\text{begin}) \}$
